



Cryptography for Developers

2026 基础技能培训第二讲

SAST

lichenghao737@gmail.com

2026-05-07

Outline



What's going to be covered:

- 对现代密码学常用技术的**使用**介绍
- 作为开发者应有的常识

What's not going to be covered:

- 密码学入门

Outline

1. Entrophy & Safety	4	Trust Wallet	10	2.4 Case Study: RFC6238 TOTP	24
1.1 Kerckhoffs' principle	6	2. Encode and Hash	13	3. Encryption	25
1.2 Security level in bits	7	2.1 Base64	15	3.1 DES, AES, Chacha20 ...	26
1.3 Entrophy in Information Theory	8	2.2 MD5, SHA, Argon2	18	3.1.1 例子: AES-CTR + MAC ...	28
1.4 BIP39 Menmonic &		2.3 Case Study: RFC 2104 HMAC	23	4. Public Key	30
				4.1 RSA	32



Outline

4.1.1 Case Study: https . . .	40	5.1.2 版本 1: SHA256	.48	5.2 雀魂与可证明的公平洗牌	.55
4.1.2 Case Study: openssh	43	5.1.3 版本 2: Salted SHA256 / HMAC_SHA256	50		
5. Examples	45	5.1.4 版本 3: bcrypt / argon2id	52		
5.1 怎样正确存储 用户登录口 令	46	5.1.5 弱点: 用 户的懒 惰	53		
5.1.1 版本 0: 明文口令 .	47				



1. Entrophy & Safety

古代的许多密码方案依赖「加密方案不为人知」



e.g.

- Pigpen Cipher A row of eight black symbols used in the Pigpen Cipher, including various geometric shapes like squares, triangles, and circles.
- Polybius A 10x10 grid of letters used in the Polybius cipher, containing the text 'GDDFAAFD' in the first row and 'GXDF' in the second row.

在现代计算机的算力面前，这些已经只作为解谜游戏的常客

1.1 Kerckhoffs' principle

1. Entrophy & Safety

The security of a cryptographic system shouldn't rely on the secrecy of the algorithm.



现代的密码方案往往采用这样的结构：

- **锁** 公开的结构（加解密算法） **可审计**
- **钥** 非公开的密钥, **可随时轮换**
- 公开通用，广受验证的锁 + 短生命周期的钥匙

1.2 Security level in bits

1. Entrophy & Safety

Defination: 一个密码方案是 n -bit 安全的, 当目前已知的 最优攻击 需要 2^n 量级的基本操作。



e.g. Bitcoin 基于的 Secp256k1 椭圆曲线公钥加密使用一个 256 bit 的私钥

- 最朴素的攻击: 把 2^{256} 个私钥全试一遍
- 已知的最优攻击 Pollard's rho algorithm : 需要 2^{128} 次操作, 从公钥算出私钥

通常认为 Secp256k1 是 128-bit 安全的。

1.2 Security level in bits

1. Entrophy & Safety

Defination: 一个密码方案是 n -bit 安全的, 当目前已知的 最优攻击 需要 2^n 量级的基本操作。



e.g. Bitcoin 基于的 Secp256k1 椭圆曲线公钥加密使用一个 256 bit 的私钥

- 最朴素的攻击: 把 2^{256} 个私钥全试一遍
- 已知的最优攻击 Pollard's rho algorithm : 需要 2^{128} 次操作, 从公钥算出私钥

通常认为 Secp256k1 是 128-bit 安全的。

在没有钥匙的情况下, 最好的锁匠要尝试 2^{128} 次才能打开

1.3 Entrophy in Information Theory

1. Entrophy & Safety

「信息熵」是 Shannon 给出的概念。

Definition: 对以 $P(X)$ 为概率质量函数的离散随机变量 X ,

$\mathbb{H}[X] = -\mathbb{E}[\log P(X)]$ 是它的熵。 单位: bit

- e.g. 长度 n 的随机 bit 流 (例如扔正反各 $\frac{1}{2}$ 的硬币的结果)

2^n 种可能各有 2^{-n} 的概率, 有 $-\log(2^{-n}) = n$ bits 的熵

不可能用少于 n 个 bit 来描述这个随机 bit 流



1.3 Entrophy in Information Theory

1. Entrophy & Safety

回顾之前「锁与钥匙」的比喻：

- 方案的 128-bit 安全：没有钥匙的情况下，锁有多难开
- 密钥本身的 **熵**：攻击者需要尝试的密钥空间有多大？

显然实际的安全性取决于以上二者的更小者。



1.4 BIP39 Menmonic & Trust Wallet

1. Entrophy & Safety

```
import random
import time
random.seed(time.time())
print(random.randbytes(64).hex())
```



```
18cbf21a462b9319da29d5132cf5d1f7fbdc0f5a7d8178a24a72821e5c4f1030
02a9df9be4ed8a171753a1762280bd4f395c290962ffeea65f8caaa9888db07d
```

有多少 bit 的熵?

1.4 BIP39 Menmonic & Trust Wallet

1. Entrophy & Safety

```
import random
import time
random.seed(time.time())
print(random.randbytes(64).hex())
```



```
18cbf21a462b9319da29d5132cf5d1f7fbd0f5a7d8178a24a72821e5c4f1030
02a9df9be4ed8a171753a1762280bd4f395c290962ffeea65f8caaa9888db07d
```

有多少 bit 的熵?

Observation: 如果 $Y = f(X)$, 那么 $\mathbb{H}[Y] \leq \mathbb{H}[X]$

由于 `time.time()` 大约是微秒级的精度, 如果我们对 `time.time()` 有年粒度的先验

$$\log(10^6 \times 3 \times 10^7) \approx \log(2^{45}) = 45 \text{ bit}$$

1.4 BIP39 Menmonic & Trust Wallet

1. Entrophy & Safety

Case Study: CVE-2024-23660



Background: 存在以下的确定性的函数关系

Random bits $\rightarrow$ BIP39 Menmonic $\rightarrow$ Bitcoin private key

每一步的映射至多保持熵，而不可能使熵增加！

- Binance Turst Wallet 曾用 用时间戳当熵源， 造成了 大约 \$ 8.5M 的损失。

1.4 BIP39 Menmonic & Trust Wallet

1. Entrophy & Safety

多少 bit 的 entrophy 是足够的?



- 不再安全: 64-bit
 - DES
 - $k2 - *37\#0=32$
- 主流标准: 128-bit
 - 12 长度的 BIP39 助记词
 - -Yy2C]]Bc^%3TdV&|4o
- Future-proof: 256-bit
 - 24 长度的 BIP39 助记词
 - @@}e/h)Y&\$~t*7oMk>-7CI4/0jA>UG1,AV|iT'M~Xpq^U

你的 info 口令足够强吗?

2. Encode and Hash

概念辨析：



- 编码: $\text{Encode}(x) = y, \text{Decode}(y) = x$
- 哈希: $\text{Hash}(x) = y$, Hopefully 不能从 y 还原 x
- 对称加密: $\text{Encrypt}(x, \text{KEY}) = y, \text{Decrypt}(y, \text{KEY}) = x$
- 非对称加密: $\text{Encrypt}(x, \text{KEY}_1) = y, \text{Decrypt}(y, \text{KEY}_2) = x$

2.1 Base64

2. Encode and Hash

解决的问题： 用一个「安全」的 ASCII 子集，表示任意二进制数据。

$2^6 = 64$, 用 64 字符之一 A..Za...z0...9+/- 表示 6 个 bit

• url-safe 版本: A..Za...z0...9-_


e.g. 0x59294e0b

拆成 bit, 即 01011001 00101001 01001110 00001011

按 6bit 拆分, 补 0

010110 010010 100101 001110 000010 110000

即 22 18 37 14 02 48 -> WS10Cw==

在树洞上，你经常可以看到：

Re Louis: jc4MyQl0K/m9zF8j6WrYbbNVpvQmWtltWx6wcRbu
qYcpC0nP0UiwKo9Ie0tFBV80E4qk3ugpP0FHg9eVn/5uFQs2wY
pa7MfVg78+Nll5zocJ16IdEzohT1ZcHxTjbRMykhtPhotpHv5e
YXQt2xM6Lxs6PAxBF0poEP8R7IxBVL4=



上述经 RSA 加密的密文本是二进制数据，不方便以文本方式传播。
实践中常以 Base64 形式表示。

2.1 Base64

2. Encode and Hash

用 base64 来编码公钥 (to be explained later)



```
curl https://github.com/lethe10137.keys
```

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDNs/uoDY57KZm81D/Tc2HDvbXSDdvj8q5CdCCEankdd
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOUSxVqUcdRS7vhFRHSRIjdbSMdmWAGofHhBzTa26Udy
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGXKNKzn4gJlQNV0XVYgztVUCnWTv2Q9W9vD0YMfmhgz
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHmoYIEgGa1jNuzQxM0pkt/y7XjsuPI/BZpdKMAqFofm
```

Base64 只是编码，不是加密！

2.2 MD5, SHA, Argon2

2. Encode and Hash

$\text{Hash}(x) = y$, Hopefully 不能从 y 还原 x

哈希，也称 **消息摘要**，希望对于任意长度的输入，给出一个固定长度的输出。且

- 「**密码学安全**」

- Preimage resistance,
- Second preimage resistance,
- Collision resistance



2.2 MD5, SHA, Argon2

2. Encode and Hash

从工程上，我们希望哈希函数

- 确定性
- 输出均匀
- 雪崩效应：



▶ `SHA256("天机常隐谁能算") =`

`e37683e38429d4179fc36e71016a55ad50b2e136d442d1fb62b481318b1ef973`

▶ `SHA256("天机 常隐谁能算") =`

`f15f337850a88aacf3e399bb64f51a69c22c20e1154cb04e6581701f878a73d3`

2.2 MD5, SHA, Argon2

2. Encode and Hash

- MD5、SHA-1 是经典算法，但已被证明不再安全 (Google: 王小云 +MD5)



工程上，我们并不关心这些算法怎么实现：

- 通常用 **SHA-256** (属于 SHA-2 家族)
- Future proof: SHA-3
- 计算大文件：BLAKE3 (Google: Merkle Tree BLAKE3)
- 「慢哈希」： **Argon2**
 - 专为密码存储设计： 计算必须消耗大量内存和时间，抗 GPU/ASIC

密码学理想模型： **单向函数** (not to be covered)

应用：

- 检查完整性



- e.g. 将大文件在机器间复制后，是否出错而内容不一致？ 比较其 SHA256/BLAKE3 而不用逐字节比较

- commit

- 如果对方能在 t_0 给出一个文件的 hash, 而在 t_1 时公布这个文件且 hash 校验通过
 - 我们可以相信对方在 t_0 之前就生成了这个文件！
 - 上机考试传录屏

应用(续):

- 派生密钥
 - 用密码学安全的 Hash 函数，把来自真实世界的长度不定的熵转成特定长度
- 存储密码（见后）
- HMAC



This definition is taken from RFC 2104:

$$\text{HMAC}(K, m) = \text{H} \left((K' \oplus \text{opad}) \parallel \text{H} \left((K' \oplus \text{ipad}) \parallel m \right) \right)$$
$$K' = \begin{cases} \text{H}(K) & \text{if } K \text{ is larger than block size} \\ K & \text{otherwise} \end{cases}$$

- 作用： 确保消息 m 的完整性， 且只有只有密钥 K 的人才能计算/校验这个 Hash

2.4 Case Study: RFC6238 TOTP

2. Encode and Hash

常见的 2FA 方式。(info.tsinghua.edu.cn 也支持!)

客户端和服务端共享一个 secret(常见 160-bit)。

- 天知地知你知我知的 K
- 两边共享 30s 粒度的时钟 T

$$\text{TOTP} = \text{Truncate}(\text{HMAC}(K, T))$$

与手机验证码对比（都是短时间有效的 6 位数）：

- 不依赖第三方（电信网络）的安全性！



3. Encryption



3.1 DES, AES, Chacha20

$\text{Encrypt}(x, \text{KEY}) = y, \text{Decrypt}(y, \text{KEY}) = x$

持有特定长度 KEY 的情况下,

- 输入: 任意长度的明文 x
- 输出: 加密后的 y , 且长度相当而一定不短于 x !



3.1 DES, AES, Chacha20

- DES: 被 AES 替代 (56-bit 密钥)
- **AES**: 密钥长度 128, 192, 256 bit
 - AES-128 + SHA-256 (128 bit 安全性)
 - AES-256 + SHA-512 (256 bit 安全性)
 - 首选: **AES-GCM / AES-CTR** 避免 ECB, CBC, CFB, OFB
- **Chacha20**:
 - 抗侧信道, 容易实现

3.1.1 例子： AES-CTR + MAC

怎样「安全」地在不安全（譬如微信）或公开（譬如树洞）信道上通信

- Confidentiality
- Integrity
- Authenticity

前提： Magically, 你和对方有 share-secret

- 对两个 256bit 长的随机 bit 串 K_1, K_2 天知地知你知我知

1. 随机生成不重复的 nonce (避免重复结构泄漏, 固定长度)
2. 计算 $\text{cipher} = \text{AES-CTR}(K_1, \text{nonce}, m)$
3. 计算 $\text{tag} = \text{HMAC_SHA256}(K_2, \text{nonce} \parallel \text{ciphertext})$
4. 发送 $\text{nonce} \parallel \text{ciphertext} \parallel \text{tag}$



对于不掌握 K_1, K_2 的攻击者:

- Confidentiality: 只要 AES-CTR 安全, 攻击者不能推出 m
- Integrity: 攻击者不能修改消息中的某部分而不被发现: 修改发送的消息中的任何部分, 会导致接收方重新计算 tag 时不匹配
- Authenticity: 只有持有 K_2 的人, 才能计算这里的 tag

4. Public Key

Problem: Key Exchange



刚刚的例子中，我们假定通信双方能安全地 share secret.

除了让双方线下接头交换密码本之外，还能怎样做？

- 一种方法是 Diffie-Hellman key exchange
- 另一种办法是基于某种非对称加密 / 公钥加密算法：

$$\text{Encrypt}(x, \text{KEY}_1) = y, \text{Decrypt}(y, \text{KEY}_2) = x$$

Question: 分解质因数:



2029226083950084700006811968448085155661237506079773379503357405500824280573454855
0831411092853265355420111327467408237607815513430169876392926664621690963444720018
1624567889349367865631271355284836138032489468762436436560544434637168644534857715
3407511577520864898452984411822860411895663876733168508594325027645076051815546542
1947478981064701338071687491705481987707786707427050792639656761228224512956160632
5084138482467434071269617902102183268707486958575273337944571523735415172542656176
3269197530658596227111884859682883889395402449770412809667301195210375732727164400
6439798021613748094639957827400320286538523

- 这个 2^{2048} 量级的整数是两个大质数之积
- 目前已知的最优算法(GNFS)的复杂度是 $O\left(\exp\left(\left(\frac{64}{9} + o(1)\right)^{\frac{1}{3}} (\log N)^{\frac{1}{3}} (\log \log N)^{\frac{2}{3}}\right)\right)$, 它 RSA-250 用了 2700 CPU 时分解一个 829-bit 的数
- 以此外推, 需要 $2700 \times \exp(26.174) \approx 6.29 \times 10^{14}$ 小时, 或 5.2 倍当前宇宙寿命的时间

计算乘法 $p \times q$:



```
p = 130577510732160665807205195583284249023683788754534299540375896156675958694193
9587136507880543439097302659607249907067302620659669248594059528885095175571317404
3996200780495444449807510794559341435641344082188671961209110443082394280626134524
9655509292158677637896083028655711083945337727596889410274071956317
q = 155403949161844088715302381602811572934727936538117248768490616594473726455689
0059721938391636129578755447601313060572520574681769411574617728826719878849162179
6929800335382682640671669102461614754130992205383368322669179747727407909142655471
5640830446326146893991957222095411447614596587990238707370860130519
```

用 python 需要数毫秒时间。

注意到 $p \times q$ 恰是上一页希望分解的数

RSA 的安全性来自于在经典计算机上，我们相信：



- 能高效地计算乘法，但不能高效地计算质因数分解
- 公钥： (n, e) e 通常取定值 65537
- 私钥： (n, d) , s.t. $(e * d) \% (p-1)(q-1) = 1$
- 加密： $c = m^e \% n$
- 解密： $m = c^d \% n$

如果你熟悉数论，以上只是欧拉定理的应用，但我们只从应用出发：



注意到：

- 公钥： (n, e) e 通常取定值 65537
- 私钥： (n, d) , s.t. $(e * d) \% (p-1)(q-1) = 1$

e 和 d 在完全对称的地位上！

- 这意味着，用其中任何一方加密，即可用另一方来解密！

考虑到用「其中任何一方加密，即可用另一方来解密」



两种使用场景：

- 传递数据： 用对方的公钥加密，对方收到后用其私钥解密
 - 即通常所言的「RSA 加密」 「RSA 解密」
- 身份证明： 用自己的私钥加密，对方收到后用你的公钥解密
 - 通常称为 「RSA 签名」 与 「RSA 验签」

Have a try:

以下的密文借由签述的 p, q 对应的 RSA 公钥加密。其中 Mode = PKCS1_OAEP, Hash = SHA256, MGFFHash = SHA256, $e=65537$



```
DWgVt7xPewDb+Q6In2U+6vHQDGZ0Z6ZrCFmvTKaw0tbIzrSjLuyzuZEnoaR+qp0i  
/g1aMNw6BxW+uir8F/p0Y0d0QgtLxNR4irTVsYTWNIpXh/0+0nNn80eB3PfeqDny  
XyaEms7kvTUQ+Leb8DtAKtzBnS/V362YNXncLEU+2m4aw6/n0+mRJjoSXmRP8xLB  
I4I236+Vz0Fhn14zXwqX0w6ww6SK1PtQqjMZofsCCzF4DQwBiteRUNU6plZT2xSa  
Rsyg+lVEIFYPNX6KMj2mp5Q2/SzjXD8CCMvKs5M2Der8SZnrA6ziWk/DoQMCD1VX  
b0pukyB58d2JRxJCsiUDEg==
```

解密以领取支付宝口令红包

Question: 有了公钥密码 (不再需要两遍先 share secret), 是否能替代对称加密?



Question: 有了公钥密码（不再需要两遍先 share secret), 是否能替代对称加密?



- 公钥密码通常计算开销较大
- 公钥密码通常并不为处理任意长度明文设计

常见 Practice:

- 传递数据：先用公钥密码加密传输一个 256-bit 的密钥，再用于对称加密
- 数字签名：
 1. 计算要生成的消息的 Hash
 2. 对 Hash，用自己的私钥加密（签名）
 3. 发送： 原消息 + Hash + 签名



4.1.1 Case Study: https

(早期的) https 正是综合利用了 RSA 的加密和签名两种用途的例子。 一个简化的流程如下：

1. Client -> Server: `client_random`
2. Server -> Client: `server_random` + X.509 证书（含 RSA 公钥）
 - 证书： 由第三方 (CA) 签发， 对 `server` 的公钥签名
3. Client 再生成一个随机数 `pre-master secret`, 以 `server` 的公钥加密后发送

后续， 基于双方已经共享的

- `client_random`
- `server_random`

- pre-master secret

生成用于 AES 等对称加密的 session key



在更现代的 TLS (TLS 1.2, TLS 1.3)下

- 基于 **椭圆曲线(Elliptic Curve)** 的非对称密码方案, 已经替代了上述的基于 RSA 的 Key Exchange 过程 (自行搜索: ECDHE)
- 目的: 如果服务器私钥泄漏了, 确保先前被录下的流量仍不能被解密 (前向安全)。

如果你这学期上《量子计算研讨课》, 会知道在 Shor 算法下, 理论上质因数分解、以及 ECC 已经不是足够难的问题 —— 如果我们有生之年能看到足够实用的量子计算机出现!

4.1.2 Case Study: openssh

无论是远程 ssh 登录，或者是在 `git.tsinghua.edu.cn` 上 clone  一个仓库，都会遇到 ssh key。

常见有两种：

- `id_rsa.pub` 与 `id_rsa` 基于 RSA 的一对公私钥
- `id_ed25519.pub` 与 `id_ed25519` 基于椭圆曲线的一对公私钥

通常，你需要把公钥 (带有 `.pub` 者) 复制到服务器上（例如，借助 `ssh-copy-id` 命令）。当你试图以公钥为凭据登录时：

1. 利用服务器的公钥，交换对称加密的密钥，建立加密信道

2. 服务器生成一个 challenge: $\text{challenge} = H(\text{session_id} || \text{user} || \text{service} || \text{pubkey} || \dots)$
3. 用私钥为这个 challenge 签名
4. 服务器验签



5. Examples

5.1 怎样正确存储用户登录口令

5. Examples

2011 年，CSDN 平台出现了 明文密码 事件。

- 「密码」作为 password 的对译容易与 encrypt 区分。以下成为「口令」
- 简而言之，它直接以明文保存了用户口令



5.1.1 版本 0: 明文口令



考虑一个登录请求：

- 用户发来(`user_id`, `password`)
- 你需要检查用户设定的口令和注册时的口令是否一致

直接明文存储口令是非常危险，但是「不是不能跑」的做法。

目标： **即使数据库泄露，攻击者也难以恢复原口令**

- 懒惰的用户很可能在许多地方使用相同的口令！ 你的泄漏可能使用户在别的网站上也被盗号。

5.1.2 版本 1: SHA256

最简单的改进： 存储 `SHA256(password)`。有些更古老的地方可能是 MD5 这样的已经不再安全的 hash 函数。



问题： 彩虹表攻击

- 本质： 攻击者预先在一个巨大的字符串集合（譬如，所有长度不超过 10 的口令）上算了一遍所有的 SHA256
- 挑战： 存储空间爆炸

5.1 怎样正确存储用户登录口令

- 彩虹表实现：存储若干链条：



- $p_0 \rightarrow h_1 \rightarrow p_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_k$

- 其中,

- $h_{n+1} = H(p_n)$, H 是某个常用的 Hash 函数

- $p_n = R(H_n)$, R 是 Reduce 函数, 用于刻意制造碰撞 (譬如, 取 Hash 的结果的前 8 个字节)

只用存储若干 (p_0, h_k) 对。

拿到一个口令的 Hash $h' = H(p')$ 后, 用相同的 R 和 H 执行相同的计算 k 轮。如果这个过程中, 出现了任何一个已知的 h_k :

- 确定了口令 p' 在某条链上! 从对应的 p_0 开始重新计算即可。

5.1.3 版本 2: Salted SHA256 / HMAC_SHA256



所谓 Salt, 就是一个随机串。

数据库里存储:

- `(user_id, salt, SHA256(salt + password))` 或
- `(user_id, salt, HMAC_SHA256(salt, password))`

对于给定的 salt, 可以认为 `Hash(password) = HMAC_SHA256(salt, password)` 是一个新的口令学安全的 Hash 函数。

- 如何解决了彩虹表攻击: 通过给每一个用户应用一个不同的 Hash 函数。攻击者不再能生成一次彩虹表, 和成千上万个用户的口令碰撞!

新的问题： 口令字典。



大多数用户是懒惰的

- 即使是 CSDN 用户，甚至是清华大学计算机系的同学
- 往往使用一些简单的常见口令，如 P@Ssw0rd

通过收集大量的口令泄漏事件中的口令，可以发现大量用户用的口令属于一个相对小的集合，即「口令字典」

- 攻击者只需要挨个试一遍就可以攻破大部分！
- By Ajax, 在 info9 禁止口令登录前，约 6 成的贵系同学的口令在常见口令字典里。

5.1.4 版本 3: `bcrypt` / `argon2id`



SHA256 有大量的商业硬件加速

- e.g. Bitcoin 矿机

一个对口令字典的缓解方案是： 让计算「加盐 Hash」变慢。 于是有了 `bcrypt`, `argon2id` 这样的「慢哈希」算法。

当前 SOTA 是 `argon2id` 算法， 它从设计上， 就为了低效：

- 需要大量的计算轮数 (如， 需要数百毫秒)
- 需要大量的内存（防止 GPU 加速）
- 限制并行计算的可能

这样， 就增大了攻击者利用口令字典来撞库的开销

5.1 怎样正确存储用户登录口令

5.1.5 弱点： 用户的懒惰

没有任何方案能解决弱口令问题



可见的未来，用户会

- 在不同的地方复用口令
- 设定自己几秒钟就能背下来的弱口令

譬如，可以检查你常用密码是否就是个弱密码：<https://haveibeenpwned.com/Passwords>

- 我们必须承认人类的懒惰。
- 因此许多地方开始启用 TOTP 等 2FA 措施

对用户的建议：

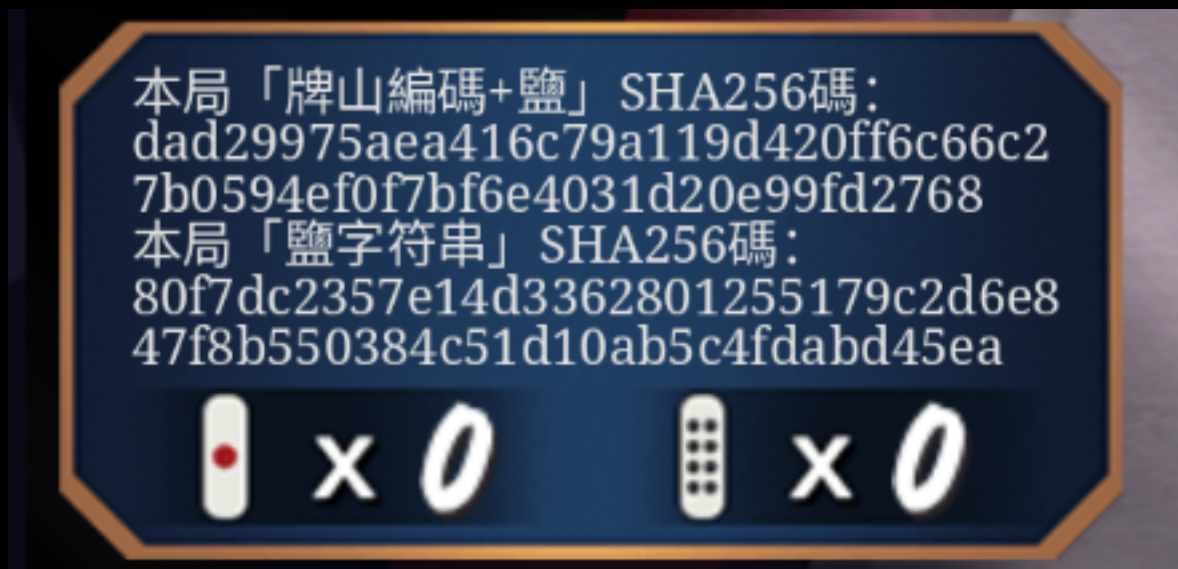
- 使用密码管理器 (Bitwarden, KeePassXC)
 - 普通人类的大脑不适合安全保存口令
- 如果一个口令在随机生成后，你 5 分钟之内能记住，大概率并不够「强」到足够作为你的密码管理器的主口令
- 永远生成随机口令：如 `pwgen -ncy 24 1`
- 唯长不破：能多长有多长。
- 即使是 16 位纯数字。也比一个长度只有 8 的混合大小写字母、符号与数字的密码更强



5.2 雀魂与可证明的公平洗牌

背景：

雀魂牌山：



5.2 雀魂与可证明的公平洗牌

雀魂只证明了：

- 自牌局开始之时起，牌山已然确定

雀魂没有证明：

- 最初确定的牌山是等概率随机生成的



5.2 雀魂与可证明的公平洗牌

- 假设： 已经通过标准方法建立了安全的信道
- 如果在不引入第三方的情况下，共同对一个随机的 bit 是 0 或 1 达成共识，且各自都不能单方面操纵结果？



5.2 雀魂与可证明的公平洗牌

5. Examples

- 假设： 已经通过标准方法建立了安全的信道
- 如果在不引入第三方的情况下，共同对一个随机的 bit 是 0 或 1 达成共识，且各自都不能单方面操纵结果？

观察： 如果双方能各自「独立」地生成一个随机 bit, 并以两个 bit 的 XOR 为结果，问题解决！

- 对每一方，只需要相信以下两点，即可确认结果是随机（0 和 1 各 0.5 概率）的
 - 自己的 bit 是随机的
 - 对方的 bit 和自己的独立

5.2 雀魂与可证明的公平洗牌

5. Examples

考虑以下的过程：

- 甲先生成一个随机数，并告知对方。
 - 乙再独立生成一个随机数，并告知对方。
1. 从乙的视角，甲生成的数是否独立于自己的？
 - 是的，因为存在以下的 happen before 关系： 甲生成随机数 $<$ 甲告知乙 $<$ 乙生成随机数
 2. 从甲的视角，乙生成的数是否独立于自己的？
 - 未必。甲无法确定乙是否是依据自己生成的数再生成了随机数



5.2 雀魂与可证明的公平洗牌

解决方案:



- 甲先生成一个 256bit 随机数 x ，并告知对方 $\text{SHA256}(x)$ 。
 - 乙再独立生成一个随机数 y ，并告知对方。
 - 甲告知乙： $x, \text{SHA256}(x)$,
 - 双方以 $x \wedge y$ 的最低位为结果
1. 从乙的视角，甲生成的数是否独立于自己的？
 - 是的，因为存在以下的 happen before 关系： $\text{甲生成 } x < \text{甲计算 } \text{SHA256}(x) < \text{甲告知乙 } \text{SHA256}(x) < \text{乙生成 } y$
 2. 从甲的视角，乙生成的数是否独立于自己的？

5.2 雀魂与可证明的公平洗牌

5. Examples

- 是的，因为存在以下的 happen before 关系：乙生成 y < 乙告知甲 y < 甲告知乙 x ，只要甲相信乙无法从 $\text{SHA256}(x)$ 中推出 x 的最低位。
- why x 要有 256 bit 而不是 1 bit? 相当于加盐

5.2 雀魂与可证明的公平洗牌

麻将牌山可以视做一个 $(0, 1, 2, \dots, 133)$ 的排列。



类似于两个 bit 计算 XOR 的性质：对于 $A \circ B$,

- 如果确信 A 是随机的(定义为 $134!$ 种可能等概率出现), 只要 B 独立于 A , 则 $A \circ B$ 也是随机的
- 如果确信 B 是随机的(定义为 $134!$ 种可能等概率出现), 只要 A 独立于 B , 则 $A \circ B$ 也是随机的

采用以下的 protocol:

1. 雀魂随机生成初始牌山 按现在的方式生成 Salted Hash, 并通知东南西北各家
2. 东南西北各家收到初始牌山的 Salted Hash 后, 各自生成一个 $(0, 1, 2, \dots, 133)$ 的排列。计算这个排列的 Salted Hash 并发送给服务器。
3. 雀魂服务器把收到的各家 Salted Hash 广播到东南西北各家
4. 东南西北各家收到其他各家的 Salted Hash 后, 把自己的排列(记做 东、南、西、北) (以及 Salt) 发给雀魂。
5. 雀魂计算 东。南。西。北。初始牌山, 开始牌局
6. 对局后, 发布以上的所有排列以及 Salted hash



5.2 雀魂与可证明的公平洗牌

5. Examples

1. 雀魂如何确认牌山按照以上的 protocol 生成?
 - 代码是雀魂写的。它可以相信自己正确实现了以上的 protocol 
2. 雀魂如何确认牌山是随机(定义为 $134!$ 种可能等概率出现)的?
 - 只需要它（如它现在声称地一样）正确随机生成了 初始牌山
 - 不需要假定玩家之间没有串通

3. 玩家如何确认 protocol 被正确执行?
 - 对局后, 比对对局前收到的其他 Salted Hash 是否与对局后公布的排列对应, 并自行计算确认牌局所用牌山计算正确。
4. 玩家如何确认牌山是随机(定义为 $134!$ 种可能等概率出现)的?
 - 只需要相信自己生成的排列是随机生成的。
 - 不需要假定雀魂和或其他玩家之间没有串通: 自己在收到了其他各方的 Salted Hash 后才发出自己的排列

总结：开发者应有的密码学常识

5. Examples

- 核心原则：不要自己发明算法 (Don't roll your own crypto)
 - 安全性不应依赖于算法的秘密性。
 - 永远优先使用经过工业界验证的标准库和算法（如 AES-GCM, Argon2id, Ed25519）。
- 木桶效应：系统安全取决于最弱的一环
 - 锁与钥匙：即便算法（锁）是 128-bit 安全的，如果密钥（钥匙）的熵不足（如使用时间戳），系统依然会崩溃。
- 承认人性的弱点：
 - 强制加盐(Salt)、使用慢哈希(Argon2id)以及推广 2FA 是开发者的基本修养。



总结：开发者应有的密码学常识

5. Examples

- 技术赋权：
 - 现代密码学给了普通人在克苏鲁面前保持隐私的可能
 - 级联 SM4 与 AES ?
- 「共识」可能不需要建立在「信任」或「权威」之上
 - 公平洗牌
 - Cryptocurrency
- Code is law, Math is Truth
- Don't trust, verify

